

Projeto 1: Traceroute Básico

Projeto 1A: Traceroute Básico

Neste projeto, você vai escrever uma implementação de traceroute em Python.

Aulas necessárias para este projeto: nenhuma. O **Guia de Traceroute** tem todas as informações que você precisa para este projeto. As Aulas 1–3 (aulas introdutórias) podem ser úteis, no entanto.

Você pode trabalhar neste projeto sozinho, ou em equipe de até 3 componentes. Lembrete: a página de políticas do curso tem uma política de colaboração que você deve seguir.

Visão Geral do Projeto

Divisão

Este projeto é dividido em duas partes: Projeto 1 e Projeto 2. Você precisa terminar o Projeto 1 antes de começar o Projeto 2.

O Projeto 1 é dividido em etapas para te guiar ao longo do projeto. Você não precisa seguir as etapas exatamente em ordem. Você pode até ignorar as etapas e simplesmente implementar o traceroute do zero, se quiser.

Teste e Depuração

Os testes do projeto são disponibilizados junto ao código e podem ser executados localmente. A avaliação possui 10 testes; o percentual de testes aprovados corresponde à nota técnica do projeto. Não há testes ocultos. A nota obtida será validada em apresentação ao professor, na qual o aluno deverá demonstrar e explicar sua implementação.

Outra forma útil de depurar é executar sua implementação de traceroute em um IP real, e ver se os resultados fazem sentido. Exemplo:

```
sudo python3 traceroute.py cmu.edu
```

Visão Geral do Código

Você vai implementar a função `traceroute` em `traceroute.py`.

Algumas funções auxiliares úteis para enviar e receber pacotes já estão implementadas em `util.py` (você pode ler esse arquivo, mas não o modifique). Também as descrevemos abaixo.

Sua Tarefa: Traceroute

Seu objetivo é implementar a função `traceroute` de forma que ela revele todos os roteadores entre o seu computador e o destino especificado.

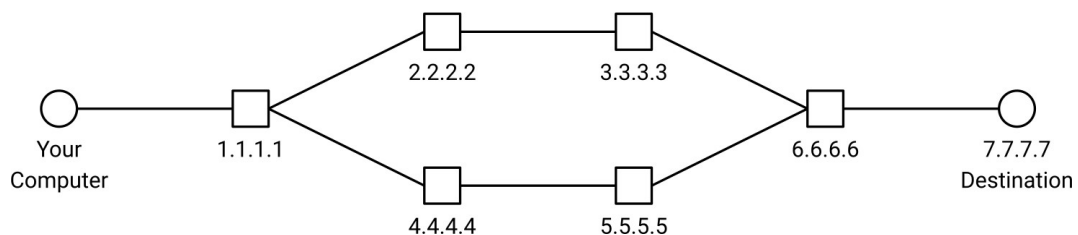
O `traceroute` recebe três argumentos:

- `ip` (string) é o destino especificado.
 - `sendsock` é um objeto que você pode usar para enviar pacotes (mais detalhes abaixo).
 - `recvsock` é um objeto que você pode usar para receber pacotes (mais detalhes abaixo).
- O `traceroute` retorna uma lista de listas, com as seguintes propriedades:
- A sublista de índice `i` contém os endereços IP de todos os roteadores que você encontrou que estão a uma distância `i+1`.
 - Por exemplo, a sublista 0 contém todos os roteadores que estão a uma distância de 1.
 - Os roteadores dentro de cada sublista podem estar em qualquer ordem.
 - Se nenhum roteador foi encontrado para uma certa distância, a sublista `i` pode ficar vazia.
 - Se `ip` for descoberto, a sublista final deve ser apenas `[ip]`.

O `traceroute` também deve chamar `util.print_result` (mais detalhes abaixo) em cada sublista, para exibir os roteadores em cada distância. Não vamos avaliar a saída impressa, mas ela é útil para depuração (além disso, fica com uma aparência legal).

Exemplo de Chamada do Traceroute

Exemplo: suponha que a topologia da rede seja assim.



Topologia com dois caminhos: do seu computador até 1.1.1.1, e então por 2.2.2.2/3.3.3.3 ou por 4.4.4.4/5.5.5.5, convergindo em 6.6.6.6 até o destino 7.7.7.7.

Supondo que nenhum erro ocorra, e que nossas sondas atinjam todos os roteadores, chamar `traceroute` com `ip=7.7.7.7` deveria retornar algo assim:

```
[["1.1.1.1"],
 ["2.2.2.2", "4.4.4.4"],
 ["3.3.3.3", "5.5.5.5"],
 ["6.6.6.6"],
 ["7.7.7.7"]]
```

Além disso, o `traceroute` deve imprimir uma saída parecida com esta:

```
traceroute to 7.7.7.7
1: 1.1.1.1
```

```
2: 2.2.2.2
   4.4.4.4
3: 3.3.3.3
   5.5.5.5
4: 6.6.6.6
5: 7.7.7.7
```

Enviando Pacotes

`sendsock` é um objeto que você pode usar para enviar pacotes de saída. Ele tem os seguintes métodos úteis:

- `sendsock.set_ttl(ttl)` define o TTL com esse número para todos os pacotes de saída subsequentes.
 - `ttl` (inteiro) é o TTL a ser definido.
- `sendsock.sendto(msg.encode(), (ip, port))` envia um pacote de saída.
 - `msg` (string) é o payload UDP do pacote. Chamamos `encode` para converter a string em bytes crus.
 - `ip` (string) é o endereço IP de destino.
 - `port` (inteiro) é a porta de

destino. Exemplo de uso:

```
# Envia um pacote "Hello" para 4.4.4.4, porta 33434, com TTL 12.
sendsock.set_ttl(12)
sendsock.sendto("Hello".encode(), ("4.4.4.4", 33434))
```

```
# Envia um pacote "Potato" para 5.5.5.5, porta 33464, com TTL 20.
sendsock.set_ttl(20)
sendsock.sendto("Potato".encode(), ("5.5.5.5", 33464))
```

Recebendo Pacotes

`recvsock` é um objeto que você pode usar para receber pacotes de entrada. Ele tem os seguintes métodos úteis:

- `recvsock.recv_select()` verifica se há algum pacote de entrada disponível para ser recebido.
 - Se houver pelo menos um pacote a ser recebido, retorna `True`.
 - Caso contrário, a função espera até que um pacote esteja disponível, ou até que um tempo limite expire.
 - Se o tempo limite expirar, retorna `False`.
- `buf, address = recvsock.recvfrom()` recebe um único pacote de entrada.
 - Retorna `buf`, os bytes crus do pacote (por exemplo, cabeçalho IPv4, seguido de um cabeçalho UDP ou ICMP, seguido do payload). Para imprimir os bytes, você pode chamar `.hex()` nos bytes crus.
 - Também retorna `address`, uma tupla contendo o endereço IP e a porta que enviaram

o pacote.

- Se não houver pacote a ser recebido, a função lança uma exceção. Por isso, você deve sempre chamar `recvsock.recv_select()` antes de chamar `recvsock.recvfrom()`.

Exemplo de uso:

```
if recvsock.recv_select(): # Verifica se ha um pacote para processar.
    buf, address = recvsock.recvfrom() # Recebe o pacote.

    # Imprime o pacote para depuracao.
    print(f"Packet bytes: {buf.hex()}")
    print(f"Packet is from IP: {address[0]}")
    print(f"Packet is from port: {address[1]}")
```

Imprimir Resultado

`print_result(routers: list[str], ttl: int)` pode ser usada para imprimir uma saída bem formatada.

Ela recebe um TTL e uma lista de endereços IP (todos os roteadores encontrados ao sondar com aquele TTL).

Sua implementação deve chamar essa função uma vez para cada TTL sondado. Exemplo

de uso:

```
util.print_result(["128.2.255.210", "128.2.42.10"], 7)
```

Isso vai fazer com que o TTL, os endereços IP de ambas as máquinas, e seus nomes correspondentes, sejam impressos de forma organizada, assim:

```
7: POD-D-DCNS-CORE2.GW.CMU.NET (128.2.255.210)
    CMU-VIP.ANDREW.CMU.EDU (128.2.42.10)
```

Etapa 1: Executar o Traceroute Manualmente

Agora que sabemos como enviar e receber pacotes, vamos tentar executar o traceroute manualmente. Não há nada para entregar nesta etapa, mas achamos que ela vai ajudar nas etapas seguintes.

Para esta etapa, você pode comentar temporariamente estas linhas do código inicial:

```
# for ttl in range(1, TRACEROUTE_MAX_TTL+1):  
#     util.print_result([], ttl)  
# return []
```

- 1 Em traceroute, escreva um código que envia um pacote para `ip` com um TTL de 1 e imprime o pacote de resposta (como bytes hexadecimais crus). O payload pode ser qualquer mensagem curta que você quiser, por exemplo, "Potato". O código inicial tem números de porta de destino que você pode usar.

- 2 Depois, tente executar esse código:

```
sudo python3 traceroute.py cmu.edu
```

- 1 Copie e cole os bytes do pacote de resposta em um decodificador de pacotes on-line como este: hpd.gasmi.net. Talvez seja necessário pedir ao decodificador para interpretar o pacote como um pacote IPv4 (já que já retiramos o cabeçalho de Camada 2).

- 2 Use o decodificador de pacotes e seu conhecimento sobre cabeçalhos para ler esse pacote. Algumas coisas para investigar:

- Quem enviou o pacote de resposta? Foi o `cmu.edu`, ou algum roteador intermediário?
- Você descobriu um roteador intermediário? Se sim, qual é o endereço IP dele? Como você descobriu esse endereço IP a partir dos cabeçalhos?
- O que o roteador intermediário está tentando te dizer? Ele está tentando reportar um erro? Se sim, como você sabe o que ele está tentando dizer?
- Quais são todos os diferentes cabeçalhos neste pacote, e por que eles estão aqui?

- 1 De volta em traceroute, tente mudar o TTL para 2, 3, 4 ou 5, e execute o código novamente para enviar o pacote com o novo TTL:

```
sudo python3 traceroute.py cmu.edu
```

Você pode ter que tentar alguns TTLs diferentes aqui, já que alguns roteadores ao longo do seu caminho até o `cmu.edu` podem não te responder.

- 1 Copie e cole os bytes do novo pacote de resposta no decodificador de pacotes, e investigue o pacote.

- Você descobriu um novo roteador intermediário?

- O que o pacote está tentando te dizer?
- 1 De volta em traceroute, tente mudar o TTL para 30. Isso deve ser saltos suficientes para alcançar o cmu.edu. Execute o código novamente para enviar o pacote com o novo TTL:

```
sudo python3 traceroute.py cmu.edu
```

- 1 Copie e cole os bytes do novo pacote de resposta no decodificador de pacotes, e investigue o pacote.
- Quem enviou o pacote de resposta? Foi o cmu.edu, ou algum roteador intermediário?
 - O que o pacote está tentando te dizer? Como você sabe?

Teste e Depuração

Executar o traceroute.py manualmente no seu computador pode ser complicado, porque você está de fato contatando roteadores reais, que podem fazer coisas estranhas.

Alguns roteadores reais que você sondar podem produzir uma saída estranha, ou nenhuma saída. Isso é esperado em redes reais. Para a avaliação, os cenários fornecidos usam uma rede simulada determinística, configurada para se comportar exatamente como descrito na especificação.

Aqui está um exemplo comum de comportamento estranho de roteadores reais que notamos. Quando você tenta decodificar um pacote, às vezes pode receber um erro dizendo que “o pacote é inválido/malformado”, assim:

! The packet is invalid/malformed : IPv4 total length exceeds packet length (56 bytes)

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
45	00	24	00	00	00	00	00	FF	01	7F	10	0A	39	94	03
0A	39	94	3F	0B	00	76	E4	00	00	00	00	45	00	00	22
AC	FC	00	00	01	11	C4	4A	0A	39	94	3F	80	02	2A	0A
C4	1C	82	9A	00	0E	37	56								

Decodificador de pacotes mostrando um campo de tamanho em little-endian, interpretado incorretamente como um valor muito grande.

No cabeçalho IPv4 deste pacote, os dois bytes correspondentes ao campo de Tamanho Total (Total Length) são 0x24 e 0x00. (Confira se você entende como encontramos esses dois bytes!)

Neste projeto, o código e os cenários de teste fornecidos esperam que esses bytes sejam enviados em big-endian, o que significa que você leria o tamanho total como $0x2400 = 9216$:

10.57.148.3 → 10.57.148.4

Internet Protocol Version 4
Total Length: 9216

! The packet is invalid. The length exceeds packet length (56 bytes)

Field offset: [2-3]
Field length: 2
Byte offset: 3

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
45	00	24	00	00	00	00	00	FF	01	7F	10	0A	39	94	03
0A	39	94	3F	0B	00	76	E4	00	00	00	00	45	00	00	22
AC	FC	00	00	01	11	C4	4A	0A	39	94	3F	80	02	2A	0A
C4	1C	82	9A	00	0E	37	56								

O mesmo campo interpretado corretamente como little-endian, resultando em um tamanho de 36 bytes.

O pacote definitivamente não tem 9216 bytes, no entanto.

Se você ler os bytes como little-endian, leria o tamanho total como $0x0024 = 36$. O payload IPv4 de fato tem 36 bytes neste pacote.

Em resumo, parece que alguns roteadores reais podem preencher o campo de Tamanho Total do IPv4 de formas diferentes:

- Alguns usam little-endian, e outros usam big-endian.
- Alguns calculam o tamanho do payload e do cabeçalho, e outros calculam apenas o tamanho do payload.

As capturas de tela acima mostram um roteador que calculou apenas o tamanho do payload, e enviou o tamanho em little-endian. Isso confundiu o decodificador de pacotes (e também confundiria seu código).

Nos cenários de teste fornecidos, os roteadores simulados sempre usam big-endian e calculam o tamanho do payload e do cabeçalho. Portanto, você não precisa tratar esses comportamentos incomuns de roteadores reais para obter a nota do projeto.

Etapa 2: Interpretando Pacotes (Parsing)

Quando você executou o traceroute manualmente, cada vez que recebia um pacote, provavelmente tinha que interpretar os bytes dos vários cabeçalhos. Sua implementação de traceroute vai precisar fazer essa interpretação em código!

Sua Tarefa

Para ajudar você a interpretar cabeçalhos em código, fornecemos três classes auxiliares, correspondentes aos três protocolos relevantes nos pacotes que você vai receber: IPv4, ICMP e UDP.

Essas classes estão sem os construtores. Sua tarefa é preencher os construtores (as funções `__init__`).

Os construtores recebem `buffer`, que são os bytes crus do cabeçalho do pacote. Por exemplo, o construtor em ICMP recebe os bytes do cabeçalho do pacote ICMP.

Seu código do construtor deve inicializar todas as variáveis de instância interpretando os bytes do pacote fornecido.

Dicas

`buffer` é um array de bytes crus, mas você vai precisar extrair bits individuais. Esta linha de código converte `buffer` em uma bitstring, `b`, na qual você pode realizar operações de string:

```
b = ''.join(format(byte, '08b') for byte in [*buffer])
```

Esta linha de código converte uma bitstring em um inteiro:

```
bitstring = '10010'          # Uma string de 1s e 0s.
number = int(bitstring, 2)    # 2, porque é um número na base 2.
print(number)                 # Imprime 18.
```

Teste e Depuração

Da Etapa 1, você já tem código que envia um pacote e recebe um pacote de resposta.

Para verificar seu código de interpretação, você pode chamar o construtor nos bytes crus que você recebe (fatando, se necessário, para extrair cabeçalhos específicos), e imprimir a saída. As classes já têm métodos `__str__` implementados para ajudar a saída a ficar com uma boa aparência.

Depois, você pode comparar a saída impressa com o decodificador de pacotes on-line para ver se você está interpretando os pacotes corretamente.

Quando seu parser estiver com uma boa aparência, execute os testes locais com `python3 avaliar.py` para verificar sua implementação.

Uma implementação correta deve passar nestes testes:

- T1. Cabeçalho IPv4 (Básico)
- T2. Cabeçalho IPv4 (com Opções)
- T3. Cabeçalho IPv4 (Campos Variados)
- T4. Cabeçalho ICMP
- T5. Cabeçalho UDP

Etapa 3: Traceroute Básico

Hora de implementar o traceroute!

Sua Tarefa

Preencha `traceroute` para descobrir todos os roteadores entre você e o destino `ip`.

Alguns lembretes:

- O payload pode ser qualquer mensagem curta que você quiser, por exemplo, "Potato".
- Para cada TTL, você deve enviar `PROBE_ATTEMPT_COUNT` pacotes com aquele TTL.
- Um IP só deve ser listado uma vez por sublista (sem duplicatas em uma sublista).
- Quando você vir uma resposta do destino (`ip`), você deve parar e retornar os roteadores que descobriu ao longo do caminho.
- Os construtores que você acabou de escrever vão ajudar a interpretar os pacotes.

Teste e Depuração

Você pode executar sua implementação para ver se sua saída se parece com a saída de um traceroute real:

```
traceroute cmu.edu # Um traceroute real.  
sudo python3 traceroute.py cmu.edu # O seu traceroute.
```

Sua saída provavelmente não vai ser exatamente igual, porque os pacotes podem viajar por caminhos diferentes cada vez que você executa o traceroute.

Quando a saída do seu traceroute estiver com uma boa aparência, execute `python3 avaliar.py` para verificar sua implementação com os testes locais.

Uma implementação correta deve passar nestes testes (além dos outros testes até agora):

- T6. Traceroute Linear
- T7. Múltiplos Caminhos
- T8. Salto sem Resposta
- T9. Remoção de Duplicatas
- T10. Parada no Destino e Impressão por TTL

Entrega e Avaliação

Entregue o arquivo `traceroute.py` conforme as orientações da disciplina no SIGAA. Os demais arquivos do pacote são auxiliares e não devem ser modificados.

A avaliação local contém 10 testes. Cada teste aprovado corresponde a 10% da nota técnica; portanto, o percentual de testes aprovados corresponde à nota do projeto (por exemplo, 80% = 8,0 em uma escala de 0 a 10). Essa nota deve ser validada em apresentação ao professor, quando o aluno deverá executar os testes e explicar as principais decisões de implementação.